

Pramo Jewels

Frontend Architecture

Document ID: PJ-FEARCH-001
Version: 1.0
Status: Draft

Field	Value
Project	Pramo Jewels Premium E-Commerce Platform
Purpose	Define the frontend architecture, structure, and engineering standards
Audience	Frontend Developers, UI Engineers, Architects, AI Coding Agents
Version	1.0

1. Frontend Vision

Build a fast, SEO-friendly, immersive and accessible luxury shopping experience using a component-driven architecture with excellent performance across desktop and mobile.

2. Recommended Technology Stack

Framework: Next.js (App Router) + React + TypeScript
Styling: Tailwind CSS
Animation: GSAP + Framer Motion
3D: Three.js + React Three Fiber + Drei
State: Zustand (UI) + TanStack Query (server state)
Forms: React Hook Form + Zod
Icons: Lucide SVG

3. Directory Structure

```
app/  
components/  
features/  
hooks/  
lib/  
services/  
store/  
styles/  
types/  
utils/  
public/
```

4. Component Architecture

Create reusable atomic components (Button, Input, Modal, Card), compose them into feature components, then build pages from those feature modules.

5. State Management

Keep UI state local where possible. Use Zustand for global UI state, TanStack Query for API caching, and URL state for filters and search.

6. Routing

Use file-based routing with nested layouts, route groups, protected account routes, dynamic product pages and SEO-friendly URLs.

7. API Communication

Consume versioned REST APIs through a centralized service layer with typed request/response models, retry policies and standardized error handling.

8. Performance

Server-side rendering where appropriate.

Image optimization.

Lazy loading.

Code splitting.

Font optimization.

Prefetch critical routes.

Target Core Web Vitals.

9. Security

Protect against XSS, sanitize inputs, store tokens securely, enforce CSP headers where possible, and never expose secrets in client-side code.

10. Accessibility

Meet WCAG 2.2 AA using semantic HTML, keyboard navigation, ARIA support, focus management and reduced-motion compatibility.

11. Testing

Use unit tests for components, integration tests for features and end-to-end tests for customer journeys. Validate responsiveness across supported browsers.

12. Coding Standards

Use TypeScript strict mode, ESLint, Prettier, reusable design tokens, descriptive naming conventions and documented component APIs.